

用 AscendC 自定义算子加速 Qwen3 微调

作者：情感机器 SwanLab 团队 AI 研究员 韩翔宇

本指导书在线版链接：[目用 AscendC 自定义算子加速 Qwen3 微调](#)

作者：

最终交付：把课程提供的 SwanRmsNorm AscendC 算子部署到 NPU + 学生自己写 PyTorch 集成层（autograd.Function + monkey-patch）+ Qwen3-8B 推理 / LoRA 微调的端到端速度对比图

注意，本实训的算子源码 + 编译脚手架在独立仓库

https://gitcode.com/pescn/RMS_Norm。

本实验你需要 **读懂源码、编译部署到 NPU、写 PyTorch 集成层、跑端到端实验、分析数据**。由于课程内容限制，算子本身**不需要从零写**，重点是掌握「自己写一个 NPU 算子并接到大模型推理/训练里」的完整链路。

1. 实验背景介绍

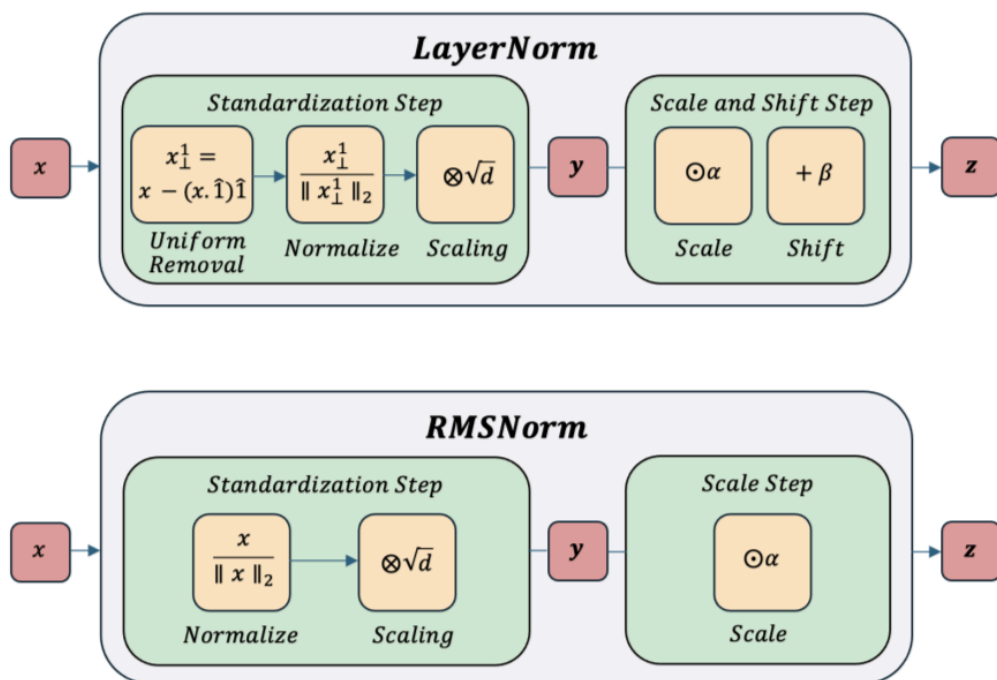
1.1 RMSNorm 算子是什么

RMSNorm（Root Mean Square Layer Normalization，均方根层归一化）是一种比传统 LayerNorm 更轻量的归一化方法，被 LLaMA、Qwen、Gemma 等主流大语言模型作为标准组件使用。

对一个 hidden 维度长 N 的向量 x ，有

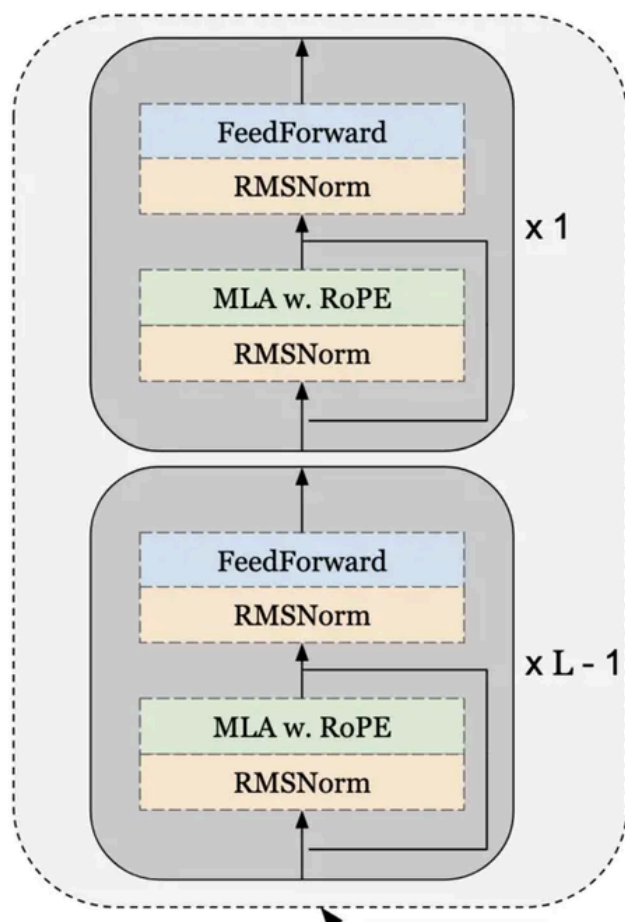
$$y_i = \frac{x_i}{\sqrt{\frac{1}{N} \sum_{j=1}^N x_j^2 + \epsilon}} \cdot \gamma_i$$

其中 γ 是可学习的缩放权重（一维）， ϵ 是数值稳定常数（典型 $1e-6$ ）。



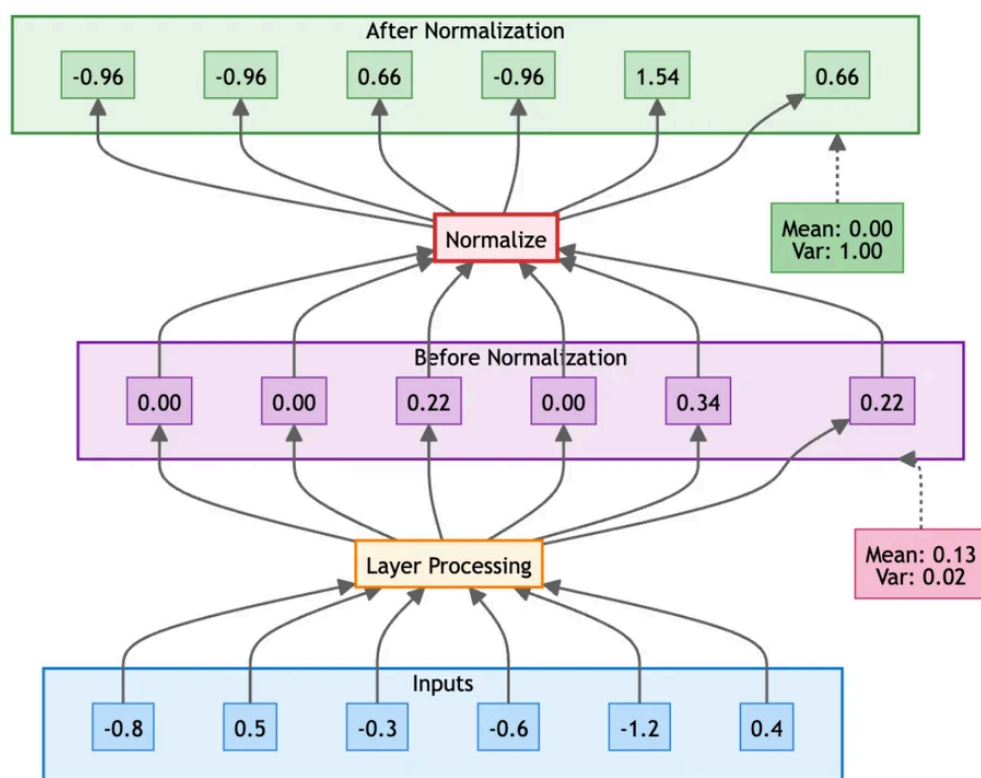
在 Transformer 中，每一层 Transformer 块里 RMSNorm 出现在 **attention 之前**和 **FFN 之前**，对输入做归一化（pre-norm 架构）。例如，在 Qwen3 中，每一层有 4 个独立的 RMSNorm：

`input_layernorm`、`post_attention_layernorm`、`q_norm`、`k_norm`（后两个是 GQA 设计中给 query 和 key 用的额外归一化），如图：



为什么模型需要它？ 神经网络深层训练中，激活值的方差会随层数累积爆炸或消失。归一化层把每一行（每个 token 的 hidden 表示）的尺度强制规整到统一的范围，让训练稳定。RMSNorm 比

LayerNorm 少一项 `mean` 减法，参数也少（只有缩放 γ ，没有偏置 β ），数学上与 LayerNorm 在大多数任务上等效，但更快。



1.2 Qwen3 中默认实现的性能开销分析

打开 transformers 源码 `transformers/models/qwen3/modeling_qwen3.py`，`Qwen3RMSNorm` 的实现是这样：

代码块

```
1 class Qwen3RMSNorm(nn.Module):
2     def __init__(self, hidden_size, eps=1e-6):
3         super().__init__()
4         self.weight = nn.Parameter(torch.ones(hidden_size))
5         self.variance_epsilon = eps
6
7     def forward(self, hidden_states):
8         input_dtype = hidden_states.dtype
9         hidden_states = hidden_states.to(torch.float32) # ①
10         variance = hidden_states.pow(2).mean(-1, keepdim=True) # ②
11         # cast bf16 → fp32
12         # pow + ③ mean
13         hidden_states = hidden_states * torch.rsqrt(variance +
14 self.variance_epsilon) # ④ +eps + ⑤ rsqrt + ⑥ mul
```

12

```
        return self.weight * hidden_states.to(input_dtype)
        cast + ⑧ broadcast mul
```

⑦

一次调用拆成 **8 个独立的 PyTorch 算子**（每个对应一次 NPU kernel launch）。每次 kernel launch 在昇腾上有 ~5-10 μ s 固定开销，加上中间结果在 HBM 之间的反复读写，**单次 RMSNorm 调用约 180-220 μ s**。

Qwen3 系列的 RMSNorm 总数：

模型	层数	每层 RMSNorm 数	总 RMSNorm 数
Qwen3-0.6B	28	4 (input + post_attn + q_norm + k_norm)	112 + 1 final = 113
Qwen3-8B (本实验)	36	4	144 + 1 = 145

每次推理 forward 调用 100+ 次 RMSNorm，单次约 200 μ s，按 Qwen3-8B 的 145 次累计 → **约 29 ms 花在 RMSNorm 上**。这部分时间占整体推理 step 时间约 **15–25%**，是值得优化的目标。

1.3 实验目标

学习内容

理解、部署并集成一个**完全融合**（fused）的 AscendC RMSNorm kernel——它把上面 8 个 PyTorch 算子合成 1 个 NPU kernel：

- 一次 kernel launch
- 一次 HBM 读 `x` 和 `y`
- 一次 HBM 写 `y`
- 全程在 UB（Unified Buffer）+ Vector unit 上完成

具体任务：

1. 读懂 `rms_norm_npu/src/` 下 4 个 AscendC 源文件 + PyTorch binding，理解 Vector unit + UB 上的 fused kernel 设计（§ 3）
2. 在远程 NPU 机器上完成编译 + 部署链路（§ 5）
3. 写一个 `torch.autograd.Function` 把 NPU kernel 包成 PyTorch 算子（§ 6.2，TODO #1）
4. 写一个 monkey-patch 函数替换 Qwen3 模型里的所有 RMSNorm 模块（§ 6.2，TODO #2）
5. 跑三个层次的端到端测试，分析数据（§ 7）

预期收益：

测量维度	数值
单算子 latency	190 μ s $\rightarrow$ 27 μ s ($\sim 7\times$ 加速)
Qwen3-8B 推理 tokens/sec	10-12 $\rightarrow$ 14-19 (1.30-1.55 $\times$ ，看卡负载)
Qwen3-8B LoRA SFT step time (B=4, S=512, r=16, AdamW)	550-580 ms $\rightarrow$ 510-520 ms (1.05-1.15 $\times$)

完成本实验后，你将：

- 掌握昇腾 AscendC 算子的完整集成链路：理解源码 $\rightarrow$ 编译 $\rightarrow$ 部署 $\rightarrow$ PyTorch 绑定 $\rightarrow$ 模型集成
- 理解 NPU Vector unit 上的 fused kernel 设计技巧
- 在端到端推理与 LoRA 微调两类负载上完成可复现的加速测量与归因分析

1.4 灵感与致谢

本课程中使用的构建脚手架来源于 GitCode 中的 xLLM 项目（Apache 2.0）

2. 算法原理

2.1 一行的处理

输入 `x.shape = [M, N]` (`M = batch  $\times$  seq_len`，`N = hidden_size`)，权重 `y.shape = [N]`。对**每一行** `x[r, :]`：

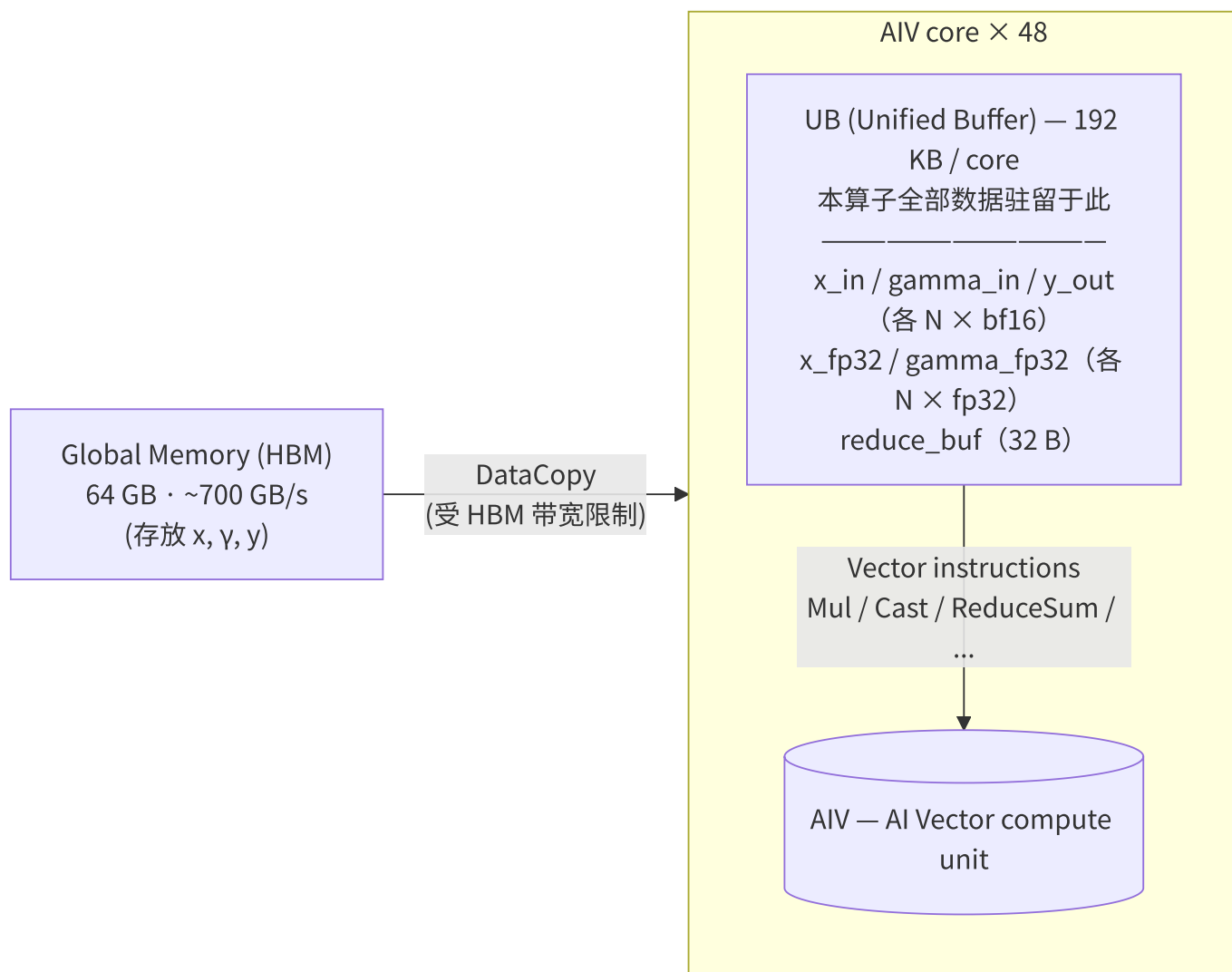
代码块

```
1  1. sumSq =  $\sum x[r, j]^2$            for j in [0, N)           # reduce 沿 N 维
2  2. mean  = sumSq / N
3  3. rstd  = 1 / sqrt(mean +  $\epsilon$ )           # 标量
4  4. y[r, j] = x[r, j]  $\cdot$  rstd  $\cdot$  y[j]   for j in [0, N) # element-wise broadcast
```

每行的计算**完全独立**——把 M 行均匀分给多个 AI Vector core 并行处理，**不需要任何核间同步**。这是 RMSNorm 在算子设计上最关键的有利特性。

2.2 从昇腾 NPU 视角分析该算子

2.2.1 昇腾 910B 的存储层次



2.2.2 UB (Unified Buffer) 是什么

UB 是 AIV core 上的 SRAM，是算子运行时唯一可控的高速片上存储。其规格如下：

维度	规格
容量	192 KB / 每个 AIV core
访问延迟	~1 cycle (vs HBM 数百 cycle)
用途	算子内部存放工作数据，避免反复读写 HBM
分配方式	通过 <code>pipe.InitBuffer(queue, depth, size)</code> 申请

为什么 RMSNorm 完全可以放进 UB： 每行只需要 N 个 $bf16$ + 一些 $fp32$ 中间数据。以 Qwen3-8B 的 `hidden=4096` 为例：

$$N \times bf16 \times 3 (x_{in}, \gamma_{in}, y_{out}) = 4096 \times 2 \times 3 = 24 \text{ KB}$$

$N \times \text{fp32} \times 2 (\text{x_fp32}, \text{gamma_fp32}) = 4096 \times 4 \times 2 = 32 \text{ KB}$
reduce 中间 $\approx 32 \text{ B}$

合计 $\approx 56 \text{ KB} \ll 192 \text{ KB} \checkmark$

即便 `hidden=12288` (Qwen3-32B) 也只用约 168 KB, 仍然装得下。

2.2.3 AIV vs AIC

昇腾 910B 上有两类计算 core:

- **AIC (AI Cube)**: 擅长矩阵乘法 (Mmad), 是 $16 \times 16 \times 16$ 的 systolic array
- **AIV (AI Vector)**: 擅长 element-wise 和 reduce 操作 (Add / Mul / ReduceSum / Cast / ...)

RMSNorm 全是 element-wise 和 reduce, **不需要 Cube**。我们的 kernel 用

`KERNEL_TYPE_AIV_ONLY` 模式, 只调度 AIV, 比 Mix 模式 (AIV + AIC) 启动更快、占用更少。

2.2.4 任务分配策略

每个 AIV core 处理 `M / 48` 行 (向上取整)。第 `core_idx` 个 core 处理 `[core_idx × rowsPerCore, (core_idx+1) × rowsPerCore)` 这个范围。例如 `M=2048`, 48 个 core, 每个处理约 43 行。

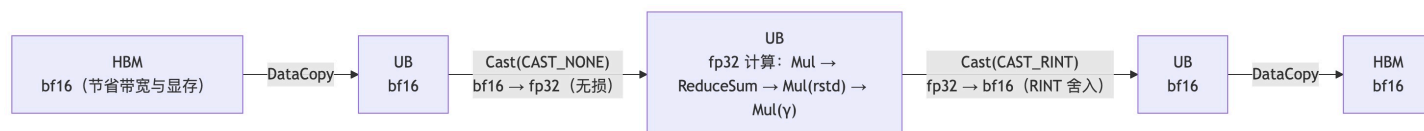
由于行间独立, **没有任何核间同步开销**——这是 RMSNorm 相对于 MatMul 在并行实现上的关键简化。

2.3 算法精度设计与分析

虽然 `x` 和 `y` 都是 `bf16` 存储, 但算子内部**累加和 reduce 必须用 fp32**。原因:

- `bf16` 只有 7 位尾数 (约 2 位十进制精度)
- $\sum x^2$ 累加 `N=4096` 次后, `bf16` 精度损失会让最终的 `mean(x2)` 误差大到不能用
- `fp32` 有 23 位尾数 (约 7 位十进制), 完全足够

精度策略:



实测精度: `bf16` 输入下, 与 PyTorch native (`fp32` 中间计算) 相比, 最大绝对误差 < 0.0625 , 相对误差 $< 1\%$ ——完全在 `bf16` 数值噪声范围内。

2.4 其他设计时的注意事项

1. **无需 pingpong 双缓冲**: RMSNorm 行间独立计算, UB 容量充裕, 单 buffer 已能满足需求。引入多缓冲只会增加实现复杂度。

2. **y 仅加载一次**：所有行共享同一份 **y**，每个 AIV core 将其从 GM 读取并 cast 成 fp32 一次后，在该 core 处理的所有行中复用。这是显著的优化点。
3. **计算 y 时需重新读取 x**：第一次 cast 后 **xFp32** 已被原地平方覆盖，得到 **rstd** 后仍需用一次原始 **x** 计算 $y = x \cdot \text{rstd} \cdot y$ ，因此会再次读取 GM。这次额外的 HBM 流量是 RMSNorm 优化中常被讨论的取舍——更高级的实现可在 UB 中保留临时副本，代价是 UB 占用翻倍。
4. **避免 `static_cast<float>(uint32_t)`**：AscendC kernel 不支持此类 cast。需在 host 端的 Tiling 函数中预先算好 $1/N$ ，并通过 TilingData 传入 kernel。

3. 算子源码分析

本章逐文件讲解 SwanRmsNorm 的 7 个源文件——全部在 `rms_norm_npu/src/`（参见 §5.1 拉取代码）。

3.1 `swan_rms_norm_tiling.h` —— TilingData 数据结构

代码块

```
1  /* AscendC RMSNorm - TilingData definition. */
2
3  #include "register/tilingdata_base.h"
4  #include "tiling/tiling_api.h"
5
6  namespace optiling {
7  BEGIN_TILING_DATA_DEF(SwanRmsNormTilingData)
8      TILING_DATA_FIELD_DEF(uint32_t, m);           // num rows
9      TILING_DATA_FIELD_DEF(uint32_t, n);           // hidden_size
10     TILING_DATA_FIELD_DEF(uint32_t, rowsPerCore);  // rows per AI core
11     TILING_DATA_FIELD_DEF(float, epsilon);          // RMSNorm epsilon
12     TILING_DATA_FIELD_DEF(float, invN);             // 1.0 / n (pre-computed)
13 END_TILING_DATA_DEF;
14
15 REGISTER_TILING_DATA_CLASS(SwanRmsNorm, SwanRmsNormTilingData)
16 }
```

作用：定义从 host 传给 kernel 的元数据结构。

`REGISTER_TILING_DATA_CLASS(SwanRmsNorm, ...)` 把这个 struct 与算子名 `SwanRmsNorm` 绑定，CANN 框架在调用时会自动序列化反序列化。

实现注意事项：`invN` 字段是必须的。AscendC kernel 不允许 `static_cast<float>(uint32_t)`，因此需要在 host 端预先算好 $1/N$ 并通过 TilingData 传入。

3.2 `swan_rms_norm_cpu.cpp` —— Tiling host + 算子注册

这个文件做两件事：

1. 实现 `TilingFunc()`：在 host 端运行，根据输入形状决定如何切分任务
2. 注册算子 `SwanRmsNorm`：声明输入输出类型、format、属性，以及该算子支持哪些芯片

3.2.1 文件头 + Tiling 函数

代码块

```
1  /* AscendC RMSNorm - Tiling host + op registration. */
2
3  #include "swan_rms_norm_tiling.h"
4  #include "register/op_def_registry.h"
5
6  namespace optiling {
7  static constexpr uint32_t NUM_CORES_AIV = 48;    // 910B 上 48 个 AIV core
8  static constexpr uint32_t N_ALIGN      = 64;    // 64 个 bf16 = 128 字节对齐
9
10 static ge::graphStatus TilingFunc(gert::TilingContext *context) {
11     SwanRmsNormTilingData tiling;
12
13     // — (1) 从 input shape 推 M 和 N —
14     auto x_shape = context->GetInputShape(0)->GetOriginShape();
15     size_t x_dim = x_shape.GetDimNum();
16     uint32_t N = x_shape.GetDim(x_dim - 1);
17     uint32_t M = 1;
18     for (size_t i = 0; i + 1 < x_dim; ++i) {
19         M *= x_shape.GetDim(i);
20     }
21
22     // — (2) 拒绝不支持的形状 (让上层 fallback 到 PyTorch native) —
23     if (M == 0 || N == 0)          return ge::GRAPH_FAILED;
24     if ((N % N_ALIGN) != 0)       return ge::GRAPH_FAILED;
25
26     // — (3) 读 epsilon (来自 OpDef 里 Attr("epsilon")) —
27     float epsilon = 1e-6f;
28     auto *attrs = context->GetAttrs();
29     if (attrs != nullptr && attrs->GetAttrNum() > 0) {
30         auto eps_attr = attrs->GetAttrPointer<float>(0);
31         if (eps_attr != nullptr) epsilon = *eps_attr;
32     }
33
34     // — (4) 任务分配: 把 M 行均分给最多 48 个 AIV core —
35     uint32_t numCores = (M < NUM_CORES_AIV) ? M : NUM_CORES_AIV;
36     uint32_t rowsPerCore = (M + numCores - 1) / numCores;
37
38     // — (5) 填 TilingData —
```

```

39     tiling.set_m(M);
40     tiling.set_n(N);
41     tiling.set_rowsPerCore(rowsPerCore);
42     tiling.set_epsilon(epsilon);
43     tiling.set_invN(1.0f / static_cast<float>(N));    // 注意: invN 在 host 算
44
45     // — (6) 启动 numCores 个 AIV core, 把 TilingData 序列化进去 —
46     context->SetBlockDim(numCores);
47     tiling.SaveToBuffer(context->GetRawTilingData()->GetData(),
48                        context->GetRawTilingData()->GetCapacity());
49     context->GetRawTilingData()->SetDataSize(tiling.GetDataSize());
50
51     // — (7) workspace 大小 (不需要额外, 但要给 lib api 默认值) —
52     size_t *currentWorkspace = context->GetWorkspaceSizes(1);
53     auto ascendcPlatform =
54         platform_ascendc::PlatformAscendC(context->GetPlatformInfo());
55     currentWorkspace[0] = ascendcPlatform.GetLibApiWorkSpaceSize();
56
57     return ge::GRAPH_SUCCESS;
58 }
59 } // namespace optiling

```

3.2.2 Shape / DataType 推导

代码块

```

1     namespace ge {
2     static ge::graphStatus InferShape(gert::InferShapeContext *context) {
3         // y 的 shape 与 x 完全相同
4         const gert::Shape *x_shape = context->GetInputShape(0);
5         gert::Shape *y_shape = context->GetOutputShape(0);
6         *y_shape = *x_shape;
7         return GRAPH_SUCCESS;
8     }
9     static ge::graphStatus InferDataType(gert::InferDataTypeContext *context) {
10        const auto inDtype = context->GetInputDataType(0);
11        context->SetOutputDataType(0, inDtype);
12        return ge::GRAPH_SUCCESS;
13    }
14 } // namespace ge

```

3.2.3 算子注册

代码块

```

1  namespace ops {
2  class SwanRmsNorm : public OpDef {
3  public:
4      explicit SwanRmsNorm(const char *name) : OpDef(name) {
5          this->Input("x")
6              .ParamType(REQUIRED)
7              .DataType({ge::DT_BF16})
8              .Format({ge::FORMAT_ND})
9              .UnknownShapeFormat({ge::FORMAT_ND});
10         this->Input("gamma")
11             .ParamType(REQUIRED)
12             .DataType({ge::DT_BF16})
13             .Format({ge::FORMAT_ND})
14             .UnknownShapeFormat({ge::FORMAT_ND});
15         this->Output("y")
16             .ParamType(REQUIRED)
17             .DataType({ge::DT_BF16})
18             .Format({ge::FORMAT_ND})
19             .UnknownShapeFormat({ge::FORMAT_ND});
20         this->Attr("epsilon").Float(1e-6f);
21
22         this->SetInferShape(ge::InferShape).SetInferDataType(ge::InferDataType);
23
24         this->AICore().SetTiling(optiling::TilingFunc);
25         this->AICore().AddConfig("ascend910b");
26         this->AICore().AddConfig("ascend910_93");
27     }
28 };
29
30 OP_ADD(SwanRmsNorm);
31 } // namespace ops

```

CANN 看到 `OP_ADD(SwanRmsNorm)` 后会**自动生成对应的 aclnn 函数** `acLnnSwanRmsNorm` (snake_case 转 camelCase 加 `acLnn` 前缀)，让我们后面在 PyTorch binding 里通过 `EXEC_NPU_CMD(acLnnSwanRmsNorm, ...)` 调用。

AddConfig 中的平台代号说明： `ascend910b` 与 `ascend910_93` 均为 CANN 内部使用的平台标识符，分别对应昇腾 910B 系列（含 910B1/B2/B3/B4 等型号）和 910_93 系列。声明这两项后，本算子在上述任一系列上都会被编译目标识别。本实验目标卡为 910B2，因此运行时实际匹配的是 `ascend910b`。

3.3 `swan\_rms\_norm.cpp` —— Kernel 实现

3.3.1 文件头 + 类成员

```

1 代码块 * AscendC RMSNorm – Vector kernel implementation. */
2
3  #include "kernel_operator.h"
4
5  using namespace AscendC;
6  using bf16 = bfloat16_t;
7
8  class KernelRmsNorm {
9  public:
10     __aicore__ inline KernelRmsNorm() {}
11
12     __aicore__ inline void Init(GM_ADDR x, GM_ADDR gamma, GM_ADDR y,
13                                GM_ADDR workspace, GM_ADDR tiling);
14     __aicore__ inline void Process();
15
16 private:
17     // — Tiling 数据 —
18     uint32_t m_, n_, rowsPC_;
19     float    epsilon_, invN_;
20     // — 当前 core 负责哪几行 —
21     int64_t  coreIdx_;
22     uint32_t rowStart_, rowEnd_;
23
24     // — UB 上的资源管理 —
25     AscendC::TPipe pipe_;
26     AscendC::GlobalTensor<bf16> xGm_, gammaGm_, yGm_;
27     AscendC::TQue<AscendC::TPosition::VECIN, 1> xQ_, gammaQ_;
28     AscendC::TQue<AscendC::TPosition::VECCOUT, 1> yQ_;
29     AscendC::TBuf<AscendC::TPosition::VECCALC> gammaFp32B_, xFp32B_, reduceB_;
30 };

```

TQue 是带 enqueue/dequeue 同步的 buffer（用于 GM↔UB 异步搬运），TBuf 是纯计算 buffer（不参与 GM 同步）。

3.3.2 Init() —— 划分 row 范围 + 申请 UB

代码块

```

1  __aicore__ inline void KernelRmsNorm::Init(GM_ADDR x, GM_ADDR gamma, GM_ADDR y,
2                                              GM_ADDR workspace, GM_ADDR tiling) {
3      GET_TILING_DATA(td, tiling);
4      m_      = td.m;
5      n_      = td.n;
6      rowsPC_ = td.rowsPerCore;
7      epsilon_ = td.epsilon;
8      invN_    = td.invN;

```

```

9
10 // — 当前 core 处理哪几行 —
11 coreIdx_ = AscendC::GetBlockIdx();
12 rowStart_ = coreIdx_ * rowsPC_;
13 rowEnd_ = rowStart_ + rowsPC_;
14 if (rowEnd_ > m_) rowEnd_ = m_;
15
16 // — GlobalTensor 指向 GM 上属于本 core 的 slice —
17 xGm_.SetGlobalBuffer((__gm__ bf16 *)x + rowStart_ * n_,
18                      (rowEnd_ - rowStart_) * n_);
19 gammaGm_.SetGlobalBuffer((__gm__ bf16 *)gamma, n_);
20 yGm_.SetGlobalBuffer((__gm__ bf16 *)y + rowStart_ * n_,
21                      (rowEnd_ - rowStart_) * n_);
22
23 // — UB 分配: x_in / y_out / gamma_in (bf16) + 两个 fp32 副本 + reduce 临时
—
24 pipe_.InitBuffer(xQ_, 1, n_ * sizeof(bf16));
25 pipe_.InitBuffer(yQ_, 1, n_ * sizeof(bf16));
26 pipe_.InitBuffer(gammaQ_, 1, n_ * sizeof(bf16));
27 pipe_.InitBuffer(gammaFp32B_, n_ * sizeof(float));
28 pipe_.InitBuffer(xFp32B_, n_ * sizeof(float));
29 pipe_.InitBuffer(reduceB_, 64);
30 }

```

`pipe_.InitBuffer(queue, depth, size)` 中 `depth=1` 表示单 buffer——RMSNorm 行间独立，不需要 pingpong 双缓冲。

3.3.3 Process() —— 主循环

👁️ AscendC 数据搬运的 5 步范式

下面的主循环里你会反复看到 `AllocTensor` → `DataCopy` → `EnQue` → `DeQue` → `计算` → `FreeTensor` 这一连串调用。这是 AscendC 的标准数据生命周期，每次"从 GM 把一段数据搬到 UB 上算"都要走完整 5 步。整个过程分别是：

1. `AllocTensor` 是在 TQue 上申请一块 UB tensor
2. `DataCopy` 从 GM 异步搬到 UB，注意这一步是 **异步** 的，因此 `DataCopy` 完直接 `Mul`，会读到没搬完的脏数据，所以下面需要有 `EnQue` 等待和 `DeQue` 阻塞
3. `EnQue` 是把 tensor 放进 queue，等待数据到达
4. `DeQue` 是阻塞直到搬运完成，拿回 tensor
5. 然后是计算部分，可以计算 `Cast` / `Mul` / `ReduceSum`
6. `FreeTensor` 最后释放占用的 UB 槽位

注意，还有如下信息：

TBuf 不参与同步，纯算时空间（如 `xFp32B_`、`gammaFp32B_`），用 `Get<T>()` 直接拿到指针即可，不需要 `EnQue/DeQue`。

PipeBarrier<PIPE_V>() 是另一种同步：Vector pipeline 内部不同指令之间偶尔需要硬性 barrier。最典型的场景是 § 3.3.3 第 ④ 步——`ReduceSum` 算完后，要在 Vector pipeline 里立即 `GetValue(0)` 取标量，必须 barrier 等 reduce 完才能读。

代码块

```
1  __aicore__ inline void KernelRmsNorm::Process() {
2      if (rowStart_ >= rowEnd_) return;
3
4      // ===== Step 0: 把  $\gamma$  读一次并 cast 到 fp32 (多行复用) =====
5      auto gammaIn = gammaQ_.AllocTensor<bf16>();
6      AscendC::DataCopy(gammaIn, gammaGm_, n_);
7      gammaQ_.EnQue(gammaIn);
8      gammaIn = gammaQ_.DeQue<bf16>();
9
10     auto gammaFp32 = gammaFp32B_.Get<float>();
11     AscendC::Cast(gammaFp32, gammaIn, AscendC::RoundMode::CAST_NONE, n_);
12     gammaQ_.FreeTensor(gammaIn);
13
14     auto xFp32      = xFp32B_.Get<float>();
15     auto reduceLoc = reduceB_.Get<float>();
16
17     // ===== 逐行处理 =====
18     for (uint32_t row = rowStart_; row < rowEnd_; ++row) {
19         uint32_t rowOff = (row - rowStart_) * n_;
20
21         // — ① 读  $x$  行 (bf16) —
22         auto xIn = xQ_.AllocTensor<bf16>();
23         AscendC::DataCopy(xIn, xGm_[rowOff], n_);
24         xQ_.EnQue(xIn);
25         xIn = xQ_.DeQue<bf16>();
26
27         // — ② Cast  $x \rightarrow \text{fp32}$  —
28         AscendC::Cast(xFp32, xIn, AscendC::RoundMode::CAST_NONE, n_);
29         xQ_.FreeTensor(xIn);
30
31         // — ③ 平方 (in-place:  $\text{xFp32} \leftarrow x^2$ ) —
32         AscendC::Mul(xFp32, xFp32, xFp32, n_);
33
34         // — ④ ReduceSum 沿  $N$  维  $\rightarrow 1$  个标量 —
35         AscendC::ReduceSum<float>(reduceLoc, xFp32, xFp32, n_);
```

```

36     AscendC::PipeBarrier<PIPE_V>(); // 等待 Vector pipe 把 ReduceSum 结
    果写入 UB 后, scalar 通道才能安全读取
37     float sumSq = reduceLoc.GetValue(0);
38
39     // — ⑤ 标量计算 rstd —
40     float mean = sumSq * invN_;
41     float rstd = 1.0f / sqrt(mean + epsilon_);
42
43     // ⚠ 第 ③ 步原地平方已经覆盖了 xFp32, 下面必须重新从 GM 读 x 并 cast 一次
44     // — ⑥ 重新读 x 并 cast —
45     auto xIn2 = xQ_.AllocTensor<bf16>();
46     AscendC::DataCopy(xIn2, xGm_[rowOff], n_);
47     xQ_.EnQueue(xIn2);
48     xIn2 = xQ_.DeQueue<bf16>();
49     AscendC::Cast(xFp32, xIn2, AscendC::RoundMode::CAST_NONE, n_);
50     xQ_.FreeTensor(xIn2);
51
52     // — ⑦  $y = x \cdot rstd \cdot y$  (全 fp32) —
53     AscendC::Mul(xFp32, xFp32, rstd, n_); //  $\times rstd$  (标量)
54     AscendC::Mul(xFp32, xFp32, gammaFp32, n_); //  $\times \gamma$  (向量)
55
56     // — ⑧ Cast fp32  $\rightarrow$  bf16, 写回 GM —
57     auto yOut = yQ_.AllocTensor<bf16>();
58     AscendC::Cast(yOut, xFp32, AscendC::RoundMode::CAST_RINT, n_);
59     yQ_.EnQueue(yOut);
60     yOut = yQ_.DeQueue<bf16>();
61     AscendC::DataCopy(yGm_[rowOff], yOut, n_);
62     yQ_.FreeTensor(yOut);
63 }
64 }

```

3.3.4 Kernel 入口

代码块

```

1  extern "C" __global__ __aicore__ void swan_rms_norm(
2      GM_ADDR x, GM_ADDR gamma, GM_ADDR y,
3      GM_ADDR workspace, GM_ADDR tiling) {
4      KERNEL_TASK_TYPE_DEFAULT(KERNEL_TYPE_AIV_ONLY); // 只调度 Vector, 不要 Cube
5      KernelRmsNorm op;
6      op.Init(x, gamma, y, workspace, tiling);
7      op.Process();
8  }

```

3.3.5 实现注意事项

- 函数名必须 `snake_case` 且与算子目录名一致： `swan_rms_norm` ↔ § 5.2 `mkdir` 的 `xllm_ops/swan_rms_norm/`，否则 CANN 自动代码生成器无法定位到对应文件。
- `PipeBarrier<PIPE_V>()` 不可省略： `ReduceSum` 是异步 Vector 指令，结果先写入 UB 但不保证 scalar register 同步可见；若不显式 barrier， `GetValue(0)` 可能读到旧值。
- `xFp32` 在第 ③ 步被原地平方覆盖：因此第 ⑥ 步必须重新从 GM 读取 `x` 并 cast 一次。这次额外的 HBM 流量是 RMSNorm 优化的常见讨论点——更高级的实现可在 UB 中保留一份临时副本，代价是 UB 占用翻倍。
- `bf16` 不直接进行 element-wise 乘法：所有运算均在 fp32 中间表示下进行，仅在 I/O 边界 cast。`bf16` 仅有 7 位尾数，N=4096 量级的累加会引入不可接受的精度损失。

3.4 `rms_norm_binding.cpp` —— PyTorch PyBind11 绑定

代码块

```

1  /* PyTorch binding for AscendC RMSNorm.
2  *
3  *      rms_norm_lib.rms_norm(x: Tensor, gamma: Tensor, eps: float) -> Tensor
4  */
5
6  #include <torch/library.h>
7  #include "pytorch_npu_helper.hpp"
8
9  static constexpr int64_t N_ALIGN = 64;
10
11 // — Fallback: 当形状/dtype 不满足时走标准 PyTorch 实现，对调用方无感 —
12 static at::Tensor py_torch_rms_norm_fallback(const at::Tensor& x,
13                                               const at::Tensor& gamma,
14                                               double eps) {
15     auto xf = x.to(torch::kFloat32);
16     auto var = xf.pow(2).mean(-1, /*keepdim=*/true);
17     auto xn = xf * at::rsqrt(var + eps);
18     return (gamma * xn.to(x.scalar_type()));
19 }
20
21 // — 主入口: dtype/shape 检查 → 调 aclnnSwanRmsNorm (或 fallback) —
22 at::Tensor rms_norm_impl_npu(const at::Tensor& x, const at::Tensor& gamma,
23                              double eps) {
24     TORCH_CHECK(x.dim() >= 2, "x must be at least 2D");
25     TORCH_CHECK(gamma.dim() == 1, "gamma must be 1D");
26     TORCH_CHECK(x.size(-1) == gamma.size(0),
27                 "x.shape[-1] must equal gamma.shape[0]");
28
29     bool dtype_ok = (x.scalar_type() == at::kBFloat16
30                     && gamma.scalar_type() == at::kBFloat16);

```

```

31     bool shape_ok = (x.size(-1) % N_ALIGN == 0);
32
33     if (!dtype_ok || !shape_ok) {
34         return py_torch_rms_norm_fallback(x, gamma, eps);
35     }
36
37     auto x_contig      = x.contiguous();
38     auto gamma_contig = gamma.contiguous();
39     at::Tensor y       = at::empty_like(x_contig);
40
41     // CANN 由 snake_case kernel 名自动生成 → aclnnSwanRmsNorm
42     EXEC_NPU_CMD(aclnnSwanRmsNorm, x_contig, gamma_contig, eps, y);
43     return y;
44 }
45
46 // — 暴露给 Python: rms_norm_lib.rms_norm(x, gamma, eps) —
47 PYBIND11_MODULE(TORCH_EXTENSION_NAME, m) {
48     m.def("rms_norm", &rms_norm_impl_npu,
49           "AscendC RMSNorm: y = (x / sqrt(mean(x²) + eps)) * gamma (bf16)");
50 }

```

这部分的作用是把 C++ 函数 `rms_norm_impl_npu` 暴露给 Python，让 `import rms_norm_lib` 后可以 `rms_norm_lib.rms_norm(x, gamma, eps)` 调用。

`EXEC_NPU_CMD` 宏由 `torch_npu` 提供，内部封装了「分配 workspace → 调用 `aclnn` API → 提交到 NPU stream」三步。

Fallback 设计：当输入不满足 `bf16 + N%64==0` 时自动回退到 PyTorch native 实现，调用方无需任何分支判断。这使得 `patch` 函数可以无差别地替换所有 RMSNorm 模块。

3.5 setup.py + 算子级 CMakeLists.txt

`src/setup.py` 把 binding 编译成 Python 扩展：

代码块

```

1  """Build script for `rms_norm_lib` Python extension."""
2  import os
3  import torch_npu
4  from setuptools import setup, find_packages
5  from torch.utils.cpp_extension import BuildExtension
6  from torch_npu.utils.cpp_extension import NpuExtension
7
8  PYTORCH_NPU_INSTALL_PATH = os.path.dirname(os.path.abspath(torch_npu.__file__))
9  USE_NINJA = os.getenv("USE_NINJA") == "1"
10 BASE_DIR = os.path.dirname(os.path.realpath(__file__))
11

```

```

12  ext = NpuExtension(
13      name="rms_norm_lib",
14      sources=[os.path.join(BASE_DIR, "rms_norm_binding.cpp)],
15      extra_compile_args=[
16          "-I" + os.path.join(PYTORCH_NPU_INSTALL_PATH,
17                              "include/third_party/acl/inc"),
18      ],
19  )
20
21  setup(
22      name="rms_norm",
23      version="1.0",
24      keywords="rms_norm",
25      ext_modules=[ext],
26      packages=find_packages(),
27      cmdclass={"build_ext": BuildExtension.with_options(use_ninja=USE_NINJA)},
28  )

```

算子级 `CMakeLists.txt` 告诉 xllm_ops 构建系统怎么编译这个算子：

代码块

```

1  add_ops_compile_options(
2      OP_NAME  SwanRmsNorm
3      OPTIONS --cce-auto-sync=on
4              -Wno-deprecated-declarations
5              -Wno-error
6  )
7
8  target_sources(op_host_aclnn PRIVATE
9      swan_rms_norm_cpu.cpp
10 )
11
12 target_sources(optiling PRIVATE
13     swan_rms_norm_cpu.cpp
14 )
15
16 target_include_directories(optiling PRIVATE
17     ${CMAKE_CURRENT_SOURCE_DIR}
18 )
19
20 target_sources(opsproto PRIVATE
21     swan_rms_norm_cpu.cpp
22 )
23
24 install(FILES swan_rms_norm.cpp

```

`OP_NAME SwanRmsNorm`（CamelCase）和算子注册时的类名一致；`target_sources` 里的 `swan_rms_norm_cpu.cpp` 和 `install FILES swan_rms_norm.cpp` 与算子目录里的实际文件名一致。

4. 实验环境准备

从本节起，所有命令需在远程实验机上**逐步执行**。每一步均有明确目的，建议先理解再执行。

4.1 基本实验环境信息

本实验在以下环境上验证通过：

项目	版本/规格
硬件	Ascend 910B2, HBM 64 GB
操作系统	EulerOS 2.0 (aarch64)
CANN	8.5.RC1（含 AscendC 编译器 <code>bisheng</code> ）
Python	3.11.14 （ system Python ，路径 <code>/usr/local/python3.11.14/bin/python3</code> ）
torch	2.9.0
torch_npu	2.9.0
transformers	≥ 4.51（Qwen3 支持需要）

关键 ENV 变量（需要 `source` 后才有）：

变量	来源	作用
<code>ASCEND_HOME_PATH</code>	<code>source /usr/local/Ascend/cann-8.5.1/bin/setenv.bash</code> 后由该脚本写入	CANN 工具链根目录
<code>ASCEND_CUSTOM_OPP_PATH</code>	<code>source ~/Ascend/custom_opp/vendors/xllm/bin/set_env.bash</code>	自定义算子部署目录（编译完算子后才有）

4.2 依赖安装

步骤 1：检查基本工具

代码块

```
1  # 1) source CANN 环境变量
2  #    注意：新开终端时 $ASCEND_HOME_PATH 是空的，必须用绝对路径 source 一次。
3  #    这一步执行后 setenv.bash 会写入 $ASCEND_HOME_PATH，之后才能用
   ${ASCEND_HOME_PATH}/...。
4  source /usr/local/Ascend/cann-8.5.1/bin/setenv.bash
5
6  # 验证 ASCEND_HOME_PATH 已正确设置
7  echo $ASCEND_HOME_PATH
8  # 应输出：/usr/local/Ascend/cann-8.5.1（或类似）
9
10 # 2) 检查 bisheng (AscendC 编译器) 是否就绪
11 which bisheng
12 # 应输出：/usr/local/Ascend/cann-8.5.1/bin/bisheng
13
14 # 3) 检查 NPU 是否可见
15 npu-smi info | head -20
16 # 应能看到 910B 卡号、HBM、温度等信息
```

关于 `${ASCEND_HOME_PATH}`：本指导书后面所有 `source`

`${ASCEND_HOME_PATH}/bin/setenv.bash` 都假设你已经在当前 shell 至少跑过一次上面的绝对路径 `source`（变量已写入）。如果换了新终端、`echo $ASCEND_HOME_PATH` 是空，回到这里重跑步骤 1。

步骤 2：给 system Python 安装 CANN 编译器需要的包

CANN 的 TBE 编译器在编译 AscendC 算子时会调用 system Python，需要一些依赖：

代码块

```
1  /usr/local/python3.11.14/bin/python3 -m pip install --user \
2      scipy decorator attrs psutil sympy cloudpickle ml-dtypes tornado absl-py
```

（若机器上已安装过，pip 会输出 "Requirement already satisfied"，可直接跳过此步。）

步骤 3：创建并激活专用 venv

为避免污染系统环境，使用独立的 Python 虚拟环境运行训练 / 推理代码：

代码块

```
1  # 基于 system Python 3.11 创建 venv，--system-site-packages 用于继承步骤 2 安装的包
```

```
2 /usr/local/python3.11.14/bin/python3 -m venv --system-site-packages
~/swan_opt_env
3
4 # 激活（每次开新终端均需执行）
5 source ~/swan_opt_env/bin/activate
6
7 # 确认 Python 来源已切换至 venv
8 which python
9 # 应输出：/home/ma-user/swan_opt_env/bin/python
```

步骤 4：在 venv 里装 PyTorch 生态

代码块

```
1 pip install --no-cache-dir \
2     torch==2.9.0 torch_npu==2.9.0 'transformers>=4.51' \
3     matplotlib accelerate safetensors modelscope peft datasets swanlab trl
```

安装大约 2 分钟。装完需要验证 NPU 和 torch_npu：

代码块

```
1 python -c "import torch, torch_npu; print('npu ok:',
2     torch.npu.is_available()); print('device:', torch_npu.npu.get_device_name(0))"
3 # 期望输出：
4 # npu ok: True
5 # device: Ascend910B2
```

4.3 模型下载

本实验使用 **Qwen3-8B**：参数规模适中，下载与启动开销小，能在端到端层面观察到 RMSNorm 优化带来的加速：

代码块

```
1 modelscope download --model Qwen/Qwen3-8B --local_dir ~/Qwen3-8B
```

4.4 把本课程仓库放到 ~/lab_rms_norm

下面 §5 / §6 / §7 全部以 `~/lab_rms_norm/` 为工作目录。如果你拿到的是 git 仓库地址：

代码块

```

1  git clone https://gitcode.com/pescn/RMS_Norm.git ~/lab_rms_norm
2  ls ~/lab_rms_norm/
3  # README.md lab_rms_norm.md lab_rms_norm.ipynb scripts/ .gitignore

```

如果是 zip / tar 压缩包，先在本地解压再 `scp -r ./lab_rms_norm ma-user@<远程机>:~/`，或者直接在远程机用 `tar xzf` 解到 `~/lab_rms_norm`。

5. 编译安装过程

课程提供的 SwanRmsNorm 算子需先编译并部署到昇腾 NPU 才能调用。本节将逐步完成编译与部署

关于代码仓：本课程把算子源码 + 编译脚手架放在独立仓库

https://gitcode.com/pescn/RMS_Norm，需要先 clone 下来（建议本地目录名改成 `rms_norm_npu` 与下面命令一致）

脚手架本身（`build_scaffold/`）来自 `xLLM-AI/xllm_ops` 的开源版本，裁剪到只有编译单算子需要的最小子集

！ 易错点：哪一步用哪个 Python？

CANN 编译算子时**必须**用 system Python（它要 import system 装的 `scipy / decorator` 等），跑训练 / 推理时**必须**用 venv 的 Python（要 `torch_npu`），两者来回切换是本章最容易踩坑的地方，对照下表执行：

步骤	Python	切换命令	原因
§ 5.3 编译算子 (<code>bash build.sh</code>)	system <code>/usr/local/python3.11.14/bin/python3</code>	<code>deactivate</code>	CANN bisheng 调 system pip 装的包
§ 5.4 部署 (<code>bash xxx.run</code>)	不调 Python	—	纯 shell
§ 5.5 source env 三件套	—	—	只 export 变量
§ 5.6 编译 PyTorch 扩展 (<code>python setup.py</code>)	venv <code>~/swan_opt_env/bin/python</code>	<code>source</code> <code>~/swan_opt_env/bin/activate</code>	需要 <code>torch_npu</code> 头/库
跑 Notebook / benchmark	venv	同上	同上

判断当前 Python 来源： `which python` —— 看到 `swan_opt_env` 路径就是 venv，看到 `/usr/local/python3.11.14/...` 就是 system。

5.1 拉取算子仓库

代码块

```
1  cd ~
2  git clone https://gitcode.com/pescn/RMS_Norm.git rms_norm_npu
3  cd ~/rms_norm_npu
4  ls
5  # README.md  src/  build_scaffold/  .gitignore
```

`src/` 下7个文件，`build_scaffold/xllm_ops/` 下只有一个 `CMakeLists.txt` ——算子目录还没创建，下一步要你自己 `mkdir + cp` 进去。

5.2 把算子源码放到脚手架对应位置

从 xllm 摘录的编译框架要求每个算子作为 `xllm_ops/<op_name>/` 子目录存在（CMake 会自动 glob 这一层）。我们的 op 名是 `swan_rms_norm`：

代码块

```
1  cd ~/rms_norm_npu
2
3  # 1) 创建算子目录
4  mkdir -p build_scaffold/xllm_ops/swan_rms_norm
5
6  # 2) 把 4 个算子文件 cp 进去 (kernel / tiling / header / 算子级 CMake)
7  cp src/swan_rms_norm.cpp \
8      src/swan_rms_norm_cpu.cpp \
9      src/swan_rms_norm_tiling.h \
10     src/CMakeLists.txt \
11     build_scaffold/xllm_ops/swan_rms_norm/
12
13  # 3) 验证
14  ls build_scaffold/xllm_ops/swan_rms_norm/
15  # CMakeLists.txt
16  # swan_rms_norm.cpp
17  # swan_rms_norm_cpu.cpp
18  # swan_rms_norm_tiling.h
```

 三处必须一致：

1. 算子目录名 `swan_rms_norm` (snake_case)
2. kernel 函数名 `extern "C" __global__ __aicore__ void swan_rms_norm(...)`

3. CMakeLists.txt 里 OP_NAME SwanRmsNorm (CamelCase)

编译过程中的自动代码生成器靠这三处对应来生成 `aclnn_swan_rms_norm.cpp`，错一个就找不到文件或链接不上

5.3 编译算子

将调用 CANN 的 `bisheng` 编译器把 AscendC 源码编译成 NPU 可执行的二进制。

CANN 编译 AscendC 算子需要 system Python（不是 venv 的 Python），它会调 § 4.2 步骤 2 给 system Python 装的 `scipy / decorator / ...`：

代码块

```
1  # 先 deactivate venv, 让 system python3 在 PATH 上
2  deactivate 2>/dev/null || true
3
4  # 确认 python3 是 system 版
5  which python3
6  # 应输出: /usr/local/python3.11.14/bin/python3
7
8  # 进入脚手架目录开始编译
9  cd ~/rms_norm_npu/build_scaffold
10 bash build.sh -n swan_rms_norm
```

`-n swan_rms_norm`：明确告诉脚手架只编译这一个算子（实际工程里 `xllm_ops` 有十几个算子时，`-n` 能把编译时间从 30 分钟缩到 1-2 分钟）。

编译大约 1-2 分钟（单算子），最后看到： `Self-extractable archive "xllm_ops.run" successfully created.` 就说明编译成功了。注意末尾可能会有几行 `Permission denied` 报错——这是 `build.sh` 默认想装到系统目录被拒，**忽略即可**，下一步我们用 `--install-path` 装到用户目录。

5.4 部署到用户目录

把编译产物（一个 `libcust_opapi.so` + JSON 元数据）放到一个 CANN 运行时能找到的目录。

CANN 默认想把算子装到 `/usr/local/Ascend/cann-8.5.1/opp/vendors/`，但这个目录是 `root` 拥有的，我们没权限。因此需要用 `--install-path` 装到用户目录：

代码块

```
1  mkdir -p ~/Ascend/custom_opp
2  bash ~/rms_norm_npu/build_scaffold/build/xllm_ops.run --install-
   path=$HOME/Ascend/custom_opp
```

`xllm_ops.run` 的输出会很长（一堆 `[ops_custom] [INFO] copy ...` 之类的日志）。**只要看最后一行：**`SUCCESS`。看到这一行就说明部署成功。如果最后没有 `SUCCESS` 而是 `[ERROR] ...`，复制整段错误回到 §8 排查表对照。

检查算子库已生成：

代码块

```
1  ls ~/Ascend/custom_opp/vendors/xllm/op_api/lib/
2  # 应该看到: libcust_opapi.so
```

5.5 设置环境变量

让 CANN 运行时知道去哪里加载我们的算子。

每次开新终端，要 `source` 三个东西才能用：

代码块

```
1  source ~/swan_opt_env/bin/activate                                # ① 激活
   venv
2  source /usr/local/Ascend/cann-8.5.1/bin/setenv.bash              # ② CANN 工
   具链（绝对路径，新 shell 必须）
3  source ~/Ascend/custom_opp/vendors/xllm/bin/set_env.bash        # ③ 自定义算
   子路径
```

为什么 ② 写绝对路径而不是 `${ASCEND_HOME_PATH}`：新开终端时 `$ASCEND_HOME_PATH` 是空的，`source ${ASCEND_HOME_PATH}/bin/setenv.bash` 会被解析成 `source /bin/setenv.bash`（不存在）。脚本本身会写入这个变量，所以先用绝对路径 `source` 一次，之后同 shell 内的命令才能用 `${ASCEND_HOME_PATH}/...`。

可以加到 `~/.bashrc` 末尾免得每次重复。

5.6 编译 PyTorch 扩展

把 C++ 端的 `rms_norm_binding.cpp`（PyTorch 与 CANN aclnn 之间的桥）编译成 Python 可以 `import` 的扩展。

代码块

```
1  cd ~/rms_norm_npu/src
2  rm -rf build dist *.egg-info      # 清理之前的产物
3  python setup.py build bdist_wheel
4  pip install --force-reinstall dist/rms_norm-*.whl
```

最后跑一个 sanity check 验证整个链路通了：

代码块

```
1  python -c "  
2  import torch, torch_npu, rms_norm_lib  
3  torch_npu.npu.set_device(0)  
4  x = torch.randn(16, 4096, dtype=torch.bfloat16, device='npu')  
5  g = torch.randn(4096, dtype=torch.bfloat16, device='npu')  
6  y = rms_norm_lib.rms_norm(x, g, 1e-6)  
7  print('shape:', y.shape, 'dtype:', y.dtype, '- OK')  
8  "  
9  # 期望输出: shape: torch.Size([16, 4096]) dtype: torch.bfloat16 - OK
```

如果走到这里没报错，算子已经可以用了。

6. PyTorch 集成与 Qwen3 接入

6.1 算子调用测试

先跑一个**精度对比**确认算子计算正确，把我们的算子结果和 PyTorch native 实现对比：

代码块

```
1  import torch, torch_npu  
2  import rms_norm_lib  
3  
4  torch_npu.npu.set_device(0)  
5  torch.manual_seed(42)  
6  
7  # Qwen3-8B 的 hidden_size = 4096  
8  M, N = 16, 4096  
9  x = torch.randn(M, N, dtype=torch.bfloat16, device='npu')  
10 gamma = torch.randn(N, dtype=torch.bfloat16, device='npu')  
11  
12 # 我们的算子  
13 y_swan = rms_norm_lib.rms_norm(x, gamma, 1e-6) # eps 用位置参数: pybind 默认未导出参数名  
14  
15 # PyTorch native (同 Qwen3RMSNorm.forward)  
16 xf = x.to(torch.float32)  
17 y_torch = (gamma * (xf * torch.rsqrt(xf.pow(2).mean(-1, keepdim=True) + 1e-6)).to(x.dtype))  
18  
19 # 对比最大绝对误差  
20 diff = (y_swan.float() - y_torch.float()).abs().max().item()
```

```
21 print(f'max |diff| = {diff:.4f}') # 期望 < 0.1
```

bf16 数值精度允许的误差量级是 $\sim 0.05-0.1$ 。

6.2 Patch Qwen3

6.2.1 Patch 的原理

我们要做的是**运行时替换**：找到模型里所有 `Qwen3RMSNorm` 模块，把它们的 `forward` 方法换成我们 `SwanRmsNorm` 的版本。这种操作俗称 **monkey-patch**。

这样我们不用动 transformers 库的源文件，未来升级 transformers 不冲突，同时一行代码就能开关，方便对比 baseline / patched，更重要的是适用于任何加载了 Qwen3 模型的 pipeline（推理、SFT、评估都行）

6.2.2 包装成 autograd.Function

如果只做推理（`model.eval()` + `torch.no_grad()`），可以直接 `module.forward = lambda x: rms_norm_lib.rms_norm(x, weight, eps)`。但**训练时反向传播**会找不到 `grad_fn`，所以要包成 `torch.autograd.Function`：

代码块

```
1 import torch
2 import torch_npu
3 import rms_norm_lib
4
5
6 class SwanRmsNormFunction(torch.autograd.Function):
7     """
8     Forward : 用我们写的 SwanRmsNorm AscendC kernel
9     Backward: 用 torch_npu 提供的 fused npu_rms_norm_backward
10              (CANN 已经把反向 kernel 写好了，直接复用，无需自己实现)
11     """
12
13     @staticmethod
14     def forward(ctx, x, gamma, eps):
15         # 1) 保证 x 内存连续 (kernel 假设 stride 对齐)
16         x_c = x.contiguous()
17
18         # 2) 调我们的 AscendC kernel 做 forward
19         y = rms_norm_lib.rms_norm(x_c, gamma, eps)
20
21         # 3) 保存 backward 需要的张量 (rstd 不存—backward 里重算更便宜)
22         ctx.save_for_backward(x_c, gamma)
23         ctx.eps = eps
```

```

24         return y
25
26     @staticmethod
27     def backward(ctx, grad_y):
28         x, gamma = ctx.saved_tensors
29         eps = ctx.eps
30
31         # 重算 rstd — 必须 fp32 (npu_rms_norm_backward 强制要求)
32         xf = x.to(torch.float32)
33         rstd = torch.rsqrt(xf.pow(2).mean(-1, keepdim=True) + eps)
34
35         # 调 torch_npu 的 fused backward kernel
36         grad_x, grad_gamma = torch_npu.npu_rms_norm_backward(
37             grad_y.contiguous(), x, gamma, rstd
38         )
39         # 返回值顺序对应 forward 的 (x, gamma, eps); eps 没有梯度 → None
40         return grad_x, grad_gamma, None

```

关键 4 点：

- `ctx.save_for_backward(x_c, gamma)` 而不是直接 `ctx.x_c = ...` ——前者会跟踪版本，后者不会，可能导致 in-place 修改后的 stale 引用。
- 为什么 **backward 重算 rstd** 而不是 **forward 时存进 ctx**：因为 `torch_npu.npu_rms_norm_backward` 把 rstd 作为 **必传参数**——它需要这个标量做反向链式求导。我们的 forward kernel 是 fused 单 launch，没有把 rstd 中间结果暴露出来；想存就得改 forward 多输出一个 `[M,1]` tensor 并 `ctx.save_for_backward(rstd)`。但 rstd 重算只是一次 per-row reduce（约 30 μ s），代价远小于“forward kernel 加输出 + 多一个 activation 占显存”。所以选择重算更划算。
- `rstd` 必须保持 fp32，**不要 `.to(bf16)`**，否则 `npu_rms_norm_backward` 直接报 dtype 错。reduce 累加 N=4096 次时 bf16 精度根本不够。
- 返回值数量必须与 `forward` 形参数量一致，无梯度的位置返回 `None`（这里 `eps` 是 float 不是 tensor，对应位置必须 None）。

6.2.3 Patch 函数

代码块

```

1  import types
2  from transformers.models.qwen3 import modeling_qwen3
3
4
5  def patch_qwen3_rmsnorm(model) -> int:
6      """

```

```

7     遍历 model 的每一个子模块，找到 Qwen3RMSNorm 实例，
8     把它的 forward 替换为调 SwanRmsNormFunction 的版本。
9
10    Returns: 被 patch 的模块数（用来确认覆盖完整）。
11    """
12    n = 0
13    for module in model.modules():
14        if isinstance(module, modeling_qwen3.Qwen3RMSNorm):
15            eps = module.variance_epsilon
16            weight = module.weight      # 这个模块自己的  $\gamma$  参数（不是全局共享！）
17
18            # — 工厂函数捕获每个模块各自的 (weight, eps) —
19            # 如果直接写 lambda x: ...weight... 会因为 Python 延迟绑定
20            # 让所有 module 都引用最后一次循环的 weight，输出乱码。
21            def make_fw(w, e):
22                def fw(self, x):
23                    return SwanRmsNormFunction.apply(x, w, e)
24                return fw
25
26            # types.MethodType 把函数绑定到 module 实例上，让它能接收 self
27            module.forward = types.MethodType(make_fw(weight, eps), module)
28            n += 1
29    return n

```

关键 2 点：

- **工厂函数 vs 闭包：** `make_fw(weight, eps)` 创建的内层 `fw` 在调用时引用的是 `make_fw` 形参 `w/e`，每次调用 `make_fw` 都是新的 frame。如果省掉工厂函数直接写 `def fw(...): return SwanRmsNormFunction.apply(x, weight, eps)`，所有 `fw` 共享外层循环的同一个 `weight` 变量名，最终全部指向最后一次的 `weight`。
- **`types.MethodType(fn, instance)`：** 把普通函数变成 bound method，让 `module.forward(x)` 能正确接收 `self`。如果直接 `module.forward = fw`，调用时 `fw(self, x)` 的 `self` 就拿不到了。

6.2.4 实际调用

代码块

```

1  import os
2  from transformers import AutoModelForCausalLM, AutoTokenizer
3
4  MODEL = os.path.expanduser('~/.Qwen3-8B')    # ~ 自动解析当前用户家目录
5  tokenizer = AutoTokenizer.from_pretrained(MODEL)
6  model = AutoModelForCausalLM.from_pretrained(MODEL,
        torch_dtype=torch.bfloat16).to('npu').eval()

```

```

7
8 # 替换前清点 Qwen3RMSNorm 模块数
9 total = sum(1 for m in model.modules() if isinstance(m,
modeling_qwen3.Qwen3RMSNorm))
10 print(f'model has {total} Qwen3RMSNorm modules')
11
12 # 执行替换
13 n = patch_qwen3_rmsnorm(model)
14 print(f'patched {n} modules') # Qwen3-8B 预期为 145
15
16 # 替换后, model 在 .generate() / .forward() 调用中所有 RMSNorm 路径均会走
SwanRmsNorm kernel

```

7. 性能提升测试

本节包含三个层次的对比测试，每项均给出参考性能数据，可用于判断算子集成是否达标。

测试基础工具：

代码块

```

1 import time
2 import torch, torch_npu
3 torch_npu.npu.set_device(0)
4
5 def bench(fn, warmup=20, repeat=200):
6     """精确测量 NPU 上一段 callable 的平均耗时（毫秒）"""
7     for _ in range(warmup):
8         fn()
9     torch_npu.npu.synchronize()
10    t0 = time.perf_counter()
11    for _ in range(repeat):
12        fn()
13    torch_npu.npu.synchronize()
14    return (time.perf_counter() - t0) / repeat * 1000.0

```

7.1 单算子性能提升测试

我们用一个典型 prefill 场景的形状测：M=128，N=4096（batch=1，prompt 128 tokens，Qwen3-8B hidden=4096）。

代码块

```

1 import rms_norm_lib
2

```

```

3  M, N = 128, 4096
4
5  def torch_rms(x, g, eps=1e-6):
6      d = x.dtype
7      xf = x.to(torch.float32)
8      return (g * (xf * torch.rsqrt(xf.pow(2).mean(-1, keepdim=True) +
eps)).to(d))
9
10 x = torch.randn(M, N, dtype=torch.bfloat16, device='npu')
11 g = torch.randn(N, dtype=torch.bfloat16, device='npu')
12
13 t_torch = bench(lambda: torch_rms(x, g))
14 t_swan = bench(lambda: rms_norm_lib.rms_norm(x, g, 1e-6))
15
16 print(f'PyTorch native : {t_torch:.3f} ms')
17 print(f'SwanRmsNorm : {t_swan:.3f} ms')
18 print(f'Speedup : {t_torch / t_swan:.2f}x')

```

参考性能（Ascend 910B2 实测）：

实现	latency	speedup
PyTorch native	~0.190 ms	1.0×
SwanRmsNorm	~0.027 ms	~7×

实测 $\text{speedup} \geq 5\times$ 即说明算子工作正常。

7.2 推理性能提升测试

测量 Qwen3-8B 端到端推理 tokens/sec。注意：baseline 与 patched 必须各自重新加载一次模型，以避免相互污染。

代码块

```

1  import os, gc
2  from transformers import AutoModelForCausalLM, AutoTokenizer
3  from transformers.models.qwen3 import modeling_qwen3
4
5  MODEL = os.path.expanduser('~/.Qwen3-8B') # 改成你下载到的路径
6  PROMPT = '请用一段话介绍量子计算的基本原理。'
7  MAX_NEW = 50
8
9  tokenizer = AutoTokenizer.from_pretrained(MODEL)
10 inputs = tokenizer(PROMPT, return_tensors='pt').to('npu')

```

```

11
12 def measure_tps(model, max_new=MAX_NEW, warmup=2):
13     for _ in range(warmup):
14         _ = model.generate(**inputs, max_new_tokens=max_new, do_sample=False,
15                             pad_token_id=tokenizer.eos_token_id)
16     torch_npu.npu.synchronize()
17     t0 = time.perf_counter()
18     out = model.generate(**inputs, max_new_tokens=max_new, do_sample=False,
19                           pad_token_id=tokenizer.eos_token_id)
20     torch_npu.npu.synchronize()
21     dt = time.perf_counter() - t0
22     return (out.shape[1] - inputs['input_ids'].shape[1]) / dt
23
24 # Baseline
25 m = AutoModelForCausalLM.from_pretrained(MODEL,
26     torch_dtype=torch.bfloat16).to('npu').eval()
27 tps_baseline = measure_tps(m)
28 print(f'Baseline : {tps_baseline:.2f} tokens/sec')
29 del m; gc.collect(); torch.npu.empty_cache()
30
31 # Patched
32 m = AutoModelForCausalLM.from_pretrained(MODEL,
33     torch_dtype=torch.bfloat16).to('npu').eval()
34 n = patch_qwen3_rmsnorm(m) # 用 6.2.3 的 patch 函数
35 tps_patched = measure_tps(m)
36 print(f'Patched ({n}) : {tps_patched:.2f} tokens/sec')
37 print(f'Speedup : {tps_patched / tps_baseline:.2f}x')

```

参考性能（Qwen3-8B, max_new_tokens=50, Ascend 910B2 实测）：

实现	tokens/sec	speedup
Baseline	~10-13	1.0×
SwanRmsNorm patched	~14-19	~1.30-1.55×

实测 baseline tokens/sec 受 prompt 长度、warmup 次数、卡负载影响，与参考值不会完全一致；但 speedup 应稳定 $\geq 1.25\times$ ，且 patched 输出文本与 baseline 完全一致（greedy decode 不改变模型输出，仅加速）。

若 speedup < 1.25×，按以下顺序排查：

- patched 模块数是否正确：** `patch_qwen3_rmsnorm(m)` 返回值应为 145（Qwen3-8B），少于此数说明 `q_norm` / `k_norm` 等支路被遗漏，未覆盖全部 RMSNorm 调用点。

2. **采样配置是否带随机性**: `do_sample=True` 会引入 sampling 开销且每次生成长度不稳定, 应使用 `do_sample=False` (greedy) 以保证可比。
3. **max_new_tokens 是否过小**: `max_new_tokens < 20` 时 prefill 占比过大, 会稀释 decode 阶段的 RMSNorm 加速; 建议保持 ≥ 50 。
4. **warmup 是否充足**: 首次 forward 包含图编译、kernel 加载等一次性开销, 至少需要 2 次 warmup; 可适当增加到 3 次。
5. **算子是否走到了 fast path**: 在 patched 路径插入断点或日志, 确认 `rms_norm_lib.rms_norm` 实际被调用, 且未触发 binding 中的 fallback 分支 (dtype 非 bf16 或 `N % 64 != 0`)。
6. **是否存在共卡负载干扰**: 使用 `npu-smi info` 确认其他进程未占用同张卡。

7.3 微调性能提升测试

测 LoRA SFT 一步 (forward + backward + optimizer.step) 的耗时。配置: `batch=4`, `seq_len=512` (M=2048)。Notebook 版本里同时把每步 `loss` 和 `step_time_ms` 用 SwanLab 记录起来——baseline 和 patched 各一个 run, 方便在 SwanLab UI 上叠加对比 loss 曲线和 step time 曲线。下面只示例核心测速代码; SwanLab 集成见 [lab_rms_norm.ipynb](#) § 7.3。

代码块

```
1  from peft import LoraConfig, get_peft_model
2
3  def build_lora(model_path):
4      base = AutoModelForCausalLM.from_pretrained(model_path,
5          torch_dtype=torch.bfloat16)
6      for p in base.parameters():
7          p.requires_grad = False
8      cfg = LoraConfig(
9          r=16, lora_alpha=32,
10         target_modules=
11         ['q_proj', 'k_proj', 'v_proj', 'o_proj', 'gate_proj', 'up_proj', 'down_proj'],
12         lora_dropout=0.0, bias='none', task_type='CAUSAL_LM',
13     )
14     return get_peft_model(base, cfg).to('npu')
15
16 def make_batch(text, seq_len=512, batch_size=4):
17     enc = tokenizer([text]*batch_size, return_tensors='pt', truncation=True,
18         padding='max_length', max_length=seq_len).to('npu')
19     labels = enc['input_ids'].clone()
20     labels[enc['attention_mask'] == 0] = -100    # 关键: 忽略 pad token, 否则
21     loss 不下降
22     enc['labels'] = labels
23     return enc
```

```

22 batch = make_batch('问：什么是机器学习？答：机器学习是 AI 的子领域，让计算机从数据中学习方法。')
23
24 def measure_step(model, batch, n_steps=15, warmup=3):
25     model.train()
26     opt = torch.optim.AdamW(
27         [p for p in model.parameters() if p.requires_grad], lr=2e-4)
28     times = []
29     for i in range(n_steps):
30         torch_npu.npu.synchronize(); t0 = time.perf_counter()
31         opt.zero_grad(set_to_none=True)
32         out = model(**batch)
33         out.loss.backward()
34         opt.step()
35         torch_npu.npu.synchronize()
36         times.append((time.perf_counter() - t0) * 1000)
37     return sum(times[warmup:]) / len(times[warmup:])
38
39 # Baseline
40 m = build_lora(MODEL)
41 t_baseline = measure_step(m, batch)
42 print(f'Baseline step time      : {t_baseline:.2f} ms')
43 del m; gc.collect(); torch.npu.empty_cache()
44
45 # Patched
46 m = build_lora(MODEL)
47 patch_qwen3_rmsnorm(m)
48 t_patched = measure_step(m, batch)
49 print(f'Patched step time       : {t_patched:.2f} ms')
50 print(f'SFT speedup            : {t_baseline / t_patched:.3f}x')

```

参考性能（Qwen3-8B, batch=4, seq=512, LoRA r=16, AdamW lr=2e-4, Ascend 910B2 实测）：

实现	step time	speedup
Baseline	~550-580 ms	1.0×
SwanRmsNorm patched	~510-520 ms	~1.05-1.15×

loss 应该从 ~3.0 单调下降到 ~2.5 左右（30 步 AdamW）。如果 loss 不动，检查 `labels[attention_mask==0]==-100` 是否到位。

SFT 加速比明显小于推理加速，原因见第九章总结部分的 Amdahl 律分析（RMSNorm 在一步 SFT 里只占 ~10%）。

8. 常见问题排查与分析

下面把实训里最常见的报错按场景分组列出，报错时按场景找到对应小节，照着改即可，如有遗漏或其他问题，也欢迎至讨论区讨论。

8.1 环境变量类（CANN / 自定义算子未 source）

8.1.1 bisheng: command not found

- **症状：**跑 `bash build.sh -n swan_rms_norm` 一开头就报 `bisheng: command not found`。
- **原因：**新终端里没 source CANN，`$ASCEND_HOME_PATH` 是空的——`bisheng` 二进制路径没进 `$PATH`。
- **解决：**必须用**绝对路径** source 一次：

代码块

```
1 source /usr/local/Ascend/cann-8.5.1/bin/setenv.bash
2 echo $ASCEND_HOME_PATH # 应输出 /usr/local/Ascend/cann-8.5.1
```

之后同 shell 内才能用 `${ASCEND_HOME_PATH}/...` 引用。

8.1.2 source: /bin/setenv.bash: No such file or directory

- **症状：**跑 `source ${ASCEND_HOME_PATH}/bin/setenv.bash` 直接报这个错
- **原因：**`$ASCEND_HOME_PATH` 为空字符串，整条命令被解析成 `source /bin/setenv.bash`
- **解决：**同 8.1.1——先用绝对路径 source 一次，让 `setenv.bash` 把变量写进当前 shell

8.1.3 aclnnSwanRmsNorm not in libopapi.so （或运行时找不到自定义算子）

- **症状：**调 `rms_norm_lib.rms_norm(...)` 报算子未注册或符号未找到。
- **原因：**算子的 `.so` 装好了但 CANN 运行时不知道去哪找——`ASCEND_CUSTOM_OPP_PATH` 没指过去。
- **解决：**

代码块

```
1 source ~/Ascend/custom_opp/vendors/xllm/bin/set_env.bash
```

如果上面这行报"找不到文件"，说明 § 5.4 部署没走完，回到 § 5.4 重跑 `bash xllm_ops.run --install-path=...`

8.2 编译期类（bisheng / TBE 报错）

8.2.1 `ModuleNotFoundError: No module named 'scipy' / 'decorator' / 'attrs'` 等

- **症状：**编译过程中 system Python 缺包报错。
- **原因：**CANN 的 TBE 编译器调 system Python（不是 venv），需要 § 4.2 步骤 2 装的依赖。
- **解决：**回到 § 4.2 步骤 2，确认这条命令跑过了：

代码块

```
1 /usr/local/python3.11.14/bin/python3 -m pip install --user \  
2     scipy decorator attrs psutil sympy cloudpickle ml-dtypes tornado absl-py
```

8.2.2 `cast between floating and unsigned integer not allowed`

- **症状：**编译 `swan_rms_norm.cpp` 报这个错。
- **原因：**kernel 里写了 `static_cast<float>(uint32_t)` ——AscendC 不支持。常见情景：在 kernel 里临时算 `1.0f / N` 这种除法。
- **解决：**把除法挪到 host 端 Tiling 函数算好，作为 `invN` 字段通过 TilingData 传进 kernel（参见 § 3.1 / § 3.2）。kernel 里只用 `Muls(..., invN_, ...)`。

8.2.3 `ubuf half *` 类型不匹配

- **症状：**bf16 上做 `Mul` / `Add` 报这个类型错。
- **原因：**bf16 在 UB 上的 element-wise 计算调度有时和 AscendC 期望的指令对不上。
- **解决：**所有 element-wise 计算放在 fp32 中间表示上做（先 `Cast(bf16 → fp32)`，算完再 `Cast(fp32 → bf16)` 写回 GM）。参见 § 3.3.3 主循环。

8.3 部署 / 安装类（权限、命名冲突）

8.3.1 `mkdir: ... opp/vendors/xllm: Permission denied`

- **症状：**跑 `bash xllm_ops.run` 默认想装到 `/usr/local/Ascend/.../opp/vendors/`，被权限拒绝。
- **原因：**该目录是 root 拥有的，普通用户写不进。
- **解决：**用 `--install-path` 装到用户目录：

代码块

```
1  mkdir -p ~/Ascend/custom_opp
2  bash build/xllm_ops.run --install-path=$HOME/Ascend/custom_opp
```

8.3.2 RuntimeError: malloc ... 107000 (部署后调用算子)

- **症状：**部署成功，调用算子时报 NPU 内存错误码 107000。
- **原因：**自定义算子名和 CANN 内置算子撞名了——CANN 把内置实现给覆盖了，但 ABI 对不上，导致 alloc 失败。
- **解决：**算子名必须独特。本课程把名字定成 SwanRmsNorm（不是 RmsNorm）正是为了避开这个冲突。如果你 fork 出去做别的算子，先 grep 一下 CANN 内置算子列表确认不冲突。

8.4 PyTorch 集成类 (autograd / monkey-patch)

8.4.1 Patch 后模型输出乱码

- **症状：**patch_qwen3_rmsnorm 跑完，model.generate() 输出全是乱码或重复符号。
- **原因：**make_fw 没用工厂函数模式，Python 闭包延迟绑定让所有 module 都引用最后一次循环的 weight，结果整个模型只用 final RMSNorm 那一份 γ 。
- **解决：**必须用工厂函数捕获——参见 § 6.2.3：

代码块

```
1  def make_fw(w, e):                                     # 工厂函数：每次调用产生新 frame
2      def fw(self, x):
3          return SwanRmsNormFunction.apply(x, w, e)
4      return fw
5  module.forward = types.MethodType(make_fw(weight, eps), module)
```

8.4.2 Backward 报 element 0 of tensors does not require grad and does not have a grad_fn

- **症状：**训练时 loss.backward() 报这个错。
- **原因：**直接调 rms_norm_lib.rms_norm(...) 没包成 torch.autograd.Function——返回的 tensor 没有 grad_fn，PyTorch autograd 接不上。
- **解决：**用 § 6.2.2 的 SwanRmsNormFunction.apply(x, gamma, eps) 替代直接调用。

8.4.3 npu_rms_norm_backward 报 dtype 错

- **症状：**backward 时 `torch_npu.npu_rms_norm_backward(...)` 报 `expected fp32 but got bf16`。
- **原因：**传进去的 `rstd` 是 bf16。这个反向 kernel 强制要求 rstd 必须 fp32（reduce 累加 N=4096 次 bf16 精度根本不够）。
- **解决：**rstd 用 fp32 重算，**不要** `.to(bf16)`。参考实现见 § 6.2.2 的 backward 函数体。

8.5 性能 / 资源类

8.5.1 推理加速比远小于参考

- **症状：**推理 speedup 跑出 $1.05\times$ 这种远低于参考的 $1.30-1.55\times$ 数字。
- **原因：**通常是 NPU 还没预热，或 `max_new_tokens` 太小让 prefill 占比过大。
- **解决：**
 - `measure_tps` 的 `warmup` 提到 ≥ 3 次
 - `max_new_tokens` 至少 50（小于 20 时 prefill 阶段稀释 decode 加速）
 - 跑前确认机器没在跑别的占卡的进程（`npu-smi info`）

8.5.2 OOM（错误码 107000，大 batch/seq 时）

- **症状：**B=8 / S=2048 之类大配置下 SFT 跑两步就 OOM。
- **原因：**activation 内存爆了——LoRA 训练里 forward 中间结果的体积约 $O(B \times S \times \text{hidden} \times \text{layers})$ 。
- **解决：**
 - 启用 gradient checkpointing：
`model.gradient_checkpointing_enable(gradient_checkpointing_kwargs={"use_reentrant": False})`
 - 或减小 batch / seq
 - 注意：开了 gradient checkpointing 后，patch 函数与 autograd.Function 的交互可能让加速比变差（实测过 $0.5\times$ ），需要单独验证

9. 总结与讨论

9.1 本实验完成的内容

本实验完整覆盖了「把 AscendC fused kernel 接到大模型推理 / 训练里」的实操链路：

1. **读懂源码：**理解 ~150 行 AscendC kernel 如何在 UB / Vector unit 上，把原本拆成 8 个 PyTorch op 的 RMSNorm 融合成 1 次 NPU 调用

2. **编译部署**：在远程机器上完成 bisheng 编译与 custom_opp 部署，将 kernel 装载到 NPU
3. **集成层实现**（学生自行编写）：用 `torch.autograd.Function` 包装 forward / backward，再以 monkey-patch + 工厂函数模式替换 Qwen3 模型中全部 145 个 RMSNorm 模块
4. **端到端测试与数据分析**：

层面	数值
单算子 latency	~7× 加速 (190 μs → 27 μs)
Qwen3-8B 推理 tokens/sec	1.30-1.55× 加速
Qwen3-8B LoRA SFT step time	1.05-1.15× 加速

9.2 为什么单算子加速 7× 而端到端只有 1.5× 或 1.1×

这是 **Amdahl 律** 的经典案例：

如果一个程序里某部分占总时间的比例是 p ，把这部分加速 s 倍后，总加速比为 $1 / (1 - p + p/s)$ 。

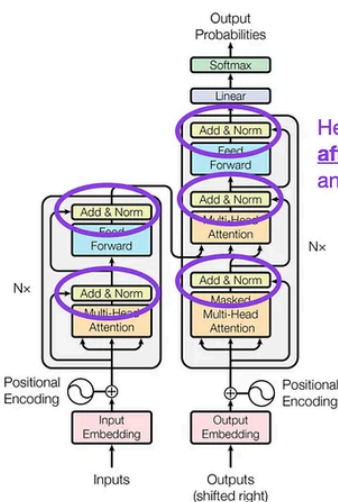
应用到我们的场景：

场景	RMSNorm 占总时间比 p	单算子加速 s	端到端加速
推理 decode	~25%	7×	~1.45×
LoRA SFT	~10%	7×	~1.08×

推理 decode 时，每生成一个 token 都要走 145 次 RMSNorm + 36 层 attention/FFN。RMSNorm 占比相对大，所以加速明显。

LoRA SFT 时，整步还包含庞大的 FFN MatMul ($M=2048 \times \text{intermediate} \times \text{hidden}$ 的矩阵乘)、attention ($O(\text{seq}^2)$)、LoRA A/B 旁路、梯度更新等，RMSNorm 只占 10% 左右。

Post-LN Transformer

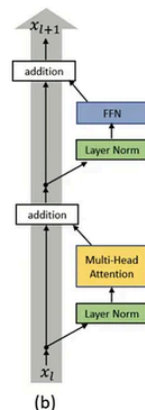
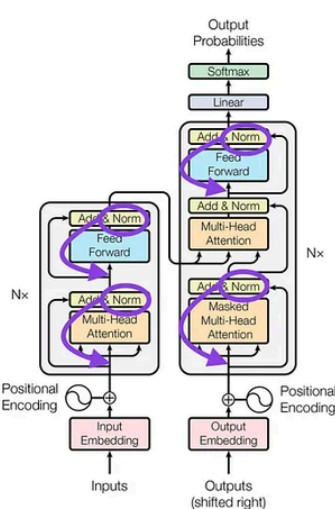


Here, LayerNorms are **after** attention and **after** fully connected layers

Better gradients if LayerNorm is placed inside residual connection, before the attention and fully connected layers

Places the layer normalization between the residual blocks: the expected gradients of the parameters near the output layer are large

Pre-LN Transformer



9.3 实践启示：识别"反复调用的小算子"并用算子优化加速

SwanRmsNorm 之所以能拿到 $1.5\times$ 端到端加速，关键在于识别出模型中被反复调用的小操作

(RMSNorm 在 Qwen3-8B 中每次 forward 被调用 145 次) 默认实现存在显著低效（被拆成了 8 个 op）。这种模式在大模型中相当常见。可以从以下这些已知的小算子 $\times$ 高频次"优化点中挑选一个，沿用本实验的同一套链路（编写 AscendC kernel $\rightarrow$ 集成 $\rightarrow$ 测速）继续优化：

- **RoPE**（旋转位置编码）：每层 attention 前都要做，PyTorch 默认实现也很碎
- **SwiGLU**（FFN 激活）：`silu(gate) * up` 也是多个 op，可以融合
- **AddRMSNorm**：residual add + RMSNorm 可以融合（torch_npu 已经提供 `npu_add_rms_norm`）
- **cast + mul + 各种 element-wise**：到处都有融合机会

算子优化的关键能力，是"识别这种小算子 $\times$ 高频次的模式"。相比从零写复杂的 MatMul，这类优化的杠杆率更高：前者能换来 $1.5\times$ 量级的端到端加速，后者通常难以胜过厂商已经深度优化的

`acInnMatmul`。本课程已经把完整的工程链路梳理清楚——从源码、编译、PyTorch 集成到端到端 benchmark，后续的算子完全可以按相同流程独立实现。

9.4 长上下文的限制

我们在 § 7.3 测的是 `batch=4, seq=512` (M=2048)。如果把 seq 进一步加大，比如做 `batch=1, seq=4096` 这种长上下文 SFT，你会发现 SwanRmsNorm 的 speedup 进一步缩小：

seq_len (batch=1)	SFT step time speedup
512	$\sim 1.28\times$
1024	$\sim 1.14\times$

2048	~1.07×
------	--------

因为 **Attention 的 compute 成本** 是 $O(\text{batch} \times \text{seq}^2 \times \text{hidden})$ ，seq 翻倍 → attention 时间 4×，但 RMSNorm 只 2×。长上下文下 attention 占总时间的比例不断上升，RMSNorm 优化的杠杆相对变小。

要在长上下文训练拿到大加速，需要写 **FlashAttention 类算子**——把 attention 的 HBM 访问压下来。这是另一个完整的算子项目，不在本课范围。

9.5 延伸思考题

- 在熟悉算子优化的完整流程后，回看自己跑的 LoRA 微调代码，有哪些**反复调用的小操作**值得做成 fused kernel？
- 如果时间充足，下一个优先优化的算子是哪个？为什么？（提示：先用 PyTorch profiler 找出占比 > 10% 的 op）
- 如果目标变成"加速长上下文训练"，写 RMSNorm 算子是否还有意义？什么算子才是关键路径？

实验结束。 把你的 `results/` 目录下的图表（推理加速 + SFT 加速）作为交付物提交到讨论区。同时请附上你的**延伸思考题答案**（简要 100-200 字即可）。